

A. Introduction

In this project, the goal is to recognize handwritten digits (from 0 to 9) using machine learning. Each digit is represented as a small 8x8 grayscale image. Recognizing digits automatically is crucial in real-world applications like:

- **Post office automation** (reading zip codes),
- **Banking systems** (processing handwritten checks),
- **Security systems** (e.g., CAPTCHA).

This task is a **multi-class classification** problem because the model must choose **one** label from **ten** possible classes (0–9).

B. Data Exploration (EDA)

Dataset Description:

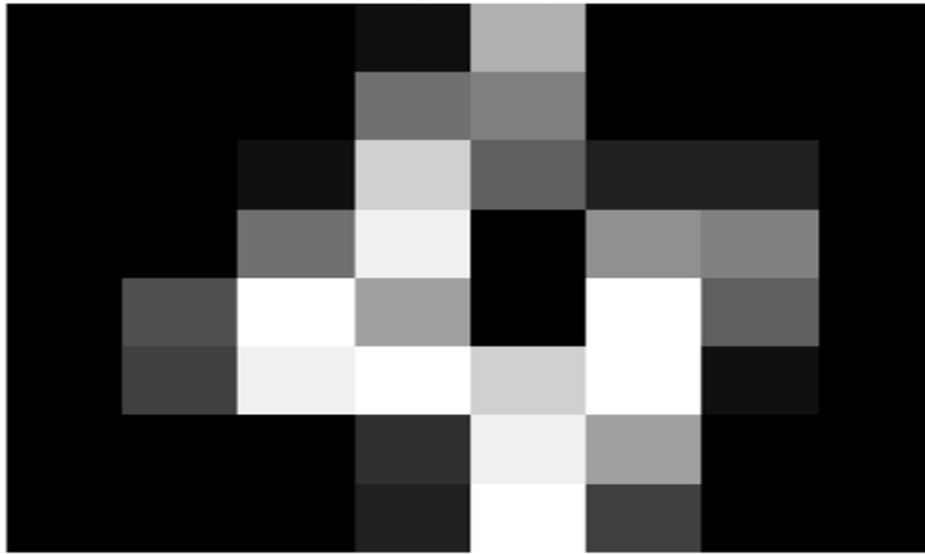
- The dataset is loaded using `sklearn.datasets.load_digits()`, which is a well-known toy dataset for testing ML algorithms.
- It has **1,797 samples** (images) and **64 features**, each representing one pixel in an 8x8 image.
- The **target variable** contains labels: 0 through 9.

Visual Exploration:

To understand the data:

- Images were displayed using `matplotlib.pyplot.imshow()`.
- The pixel values range from 0 (white) to 16 (black), representing intensity.
- Example:

Example Digit: 4



Key Observations:

- The dataset is well-balanced: each class (digit) has roughly the same number of examples.
- No missing data — this is a clean dataset.
- Because the features are pixel intensities, some algorithms (like SVM and KNN) **perform better when features are scaled** to standardize their range.

C. Methodology

Preprocessing:

To make the data suitable for modeling:

- **StandardScaler** was applied to normalize features: this ensures no pixel dominates due to scale.
- **train_test_split** divided data into:
 - **80% training** (used to train the model),
 - **20% testing** (used to evaluate generalization).

Algorithms Used:

Five models were chosen to reflect different approaches:

1. **Logistic Regression:**

- A simple linear classifier often used as a baseline.
- 2. **K-Nearest Neighbors (KNN):**
 - Classifies a point based on the “k” closest neighbors. Works well on image data with a good distance metric.
- 3. **Naive Bayes:**
 - Assumes features are independent given the class. Often fast but can struggle when features are correlated (as in images).
- 4. **Decision Tree:**
 - A model that splits the data based on feature thresholds. Can be prone to overfitting without pruning.
- 5. **Support Vector Machine (SVM):**
 - A powerful classifier that finds the best boundary between classes. Linear kernel was used here.

Each model brings a different inductive bias and is trained under the same conditions to ensure fairness in comparison.

D. Model Training

All models were trained using default or simple hyperparameters to keep comparisons straightforward:

- `LogisticRegression(max_iter=1000)` ensures convergence during optimization.
- `KNN(n_neighbors=5)` means the prediction is based on the 5 closest neighbors.
- `SVM(kernel='linear')` avoids kernel trick complexity for this simple dataset.

The training was followed by evaluation on the 20% test set (not seen during training) to estimate real-world performance.

E. Results and Evaluation

Metrics Explained:

- **Accuracy:** Percentage of correct predictions.
- **Precision:** How many predicted digits were correct (for each class).
- **Recall:** How many actual digits were correctly identified.
- **F1 Score:** Harmonic mean of precision and recall — balances both.

All scores are **macro-averaged**, meaning each class is treated equally regardless of how many samples it has.

Performance Summary (in detail):

Logistic Regression Evaluation:

Accuracy : 0.9722
Precision : 0.9737
Recall : 0.9744
F1 Score : 0.9740

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	33
1	0.97	1.00	0.98	28
2	1.00	1.00	1.00	33
3	0.97	0.97	0.97	34
4	1.00	0.98	0.99	46
5	0.94	0.94	0.94	47
6	0.97	0.97	0.97	35
7	1.00	0.97	0.99	34
8	0.97	0.97	0.97	30
9	0.93	0.95	0.94	40
accuracy			0.97	360
macro avg	0.97	0.97	0.97	360
weighted avg	0.97	0.97	0.97	360

🔍 Decision Tree Evaluation:

Accuracy : 0.8417
Precision : 0.8465
Recall : 0.8365
F1 Score : 0.8395

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.88	0.92	33
1	0.85	0.79	0.81	28
2	0.86	0.76	0.81	33
3	0.76	0.85	0.81	34
4	0.81	0.91	0.86	46
5	0.91	0.85	0.88	47
6	1.00	0.91	0.96	35
7	0.82	0.91	0.86	34
8	0.75	0.70	0.72	30
9	0.74	0.80	0.77	40
accuracy			0.84	360
macro avg	0.85	0.84	0.84	360
weighted avg	0.85	0.84	0.84	360

🔍 K-Nearest Neighbors Evaluation:

Accuracy : 0.9750
Precision : 0.9765
Recall : 0.9769
F1 Score : 0.9765

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	33
1	1.00	1.00	1.00	28
2	0.97	1.00	0.99	33
3	0.97	0.97	0.97	34
4	0.98	1.00	0.99	46
5	0.96	0.96	0.96	47
6	0.97	1.00	0.99	35
7	1.00	0.94	0.97	34
8	0.97	1.00	0.98	30
9	0.95	0.90	0.92	40
accuracy			0.97	360
macro avg	0.98	0.98	0.98	360
weighted avg	0.98	0.97	0.97	360

🔍 Naive Bayes Evaluation:

Accuracy : 0.7667
Precision : 0.8170
Recall : 0.7705
F1 Score : 0.7599

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.94	0.95	33
1	0.75	0.75	0.75	28
2	0.86	0.36	0.51	33
3	0.93	0.76	0.84	34
4	1.00	0.52	0.69	46
5	0.81	0.94	0.87	47
6	0.88	1.00	0.93	35
7	0.52	0.97	0.68	34
8	0.49	0.83	0.62	30
9	0.96	0.62	0.76	40
accuracy			0.77	360
macro avg	0.82	0.77	0.76	360
weighted avg	0.83	0.77	0.76	360

🔍 SVM (Linear Kernel) Evaluation:

Accuracy : 0.9750
Precision : 0.9753
Recall : 0.9753
F1 Score : 0.9751

Classification Report:

	precision	recall	f1-score	support
0	0.97	1.00	0.99	33
1	0.97	1.00	0.98	28
2	1.00	1.00	1.00	33
3	0.94	0.97	0.96	34
4	1.00	1.00	1.00	46
5	0.96	0.96	0.96	47
6	0.97	0.97	0.97	35
7	0.97	0.97	0.97	34
8	1.00	0.93	0.97	30
9	0.97	0.95	0.96	40
accuracy			0.97	360
macro avg	0.98	0.98	0.98	360
weighted avg	0.98	0.97	0.97	360

Model	Accuracy	Precision	Recall	F1 Score	Notes
KNN	0.975	0.976	0.977	0.976	Excellent for this dataset; distance-based voting is effective.
SVM (Linear)	0.975	0.975	0.975	0.975	Consistent and reliable; margin-based optimization works well.
Logistic Regression	0.972	0.974	0.974	0.974	Strong baseline; performs close to SVM.
Decision Tree	0.842	0.846	0.837	0.840	Simple tree overfits; struggles on ambiguous digits.
Naive Bayes	0.767	0.817	0.770	0.760	Performs worst; assumes feature independence, which is unrealistic for pixel data.

Confusion Matrix Insights:

- Helps pinpoint specific digits where models fail.
- Example issues:
 - Naive Bayes misclassifies 5 as 3 or 8 frequently.
 - Decision Tree has more random-looking misclassifications.

F. Conclusion

Summary:

- **KNN and SVM** tied with the highest accuracy (97.5%).
- **Logistic Regression** also performed very well and can be a solid choice when speed matters.
- **Naive Bayes** underperforms due to unrealistic assumptions for image data.
- **Decision Tree** struggles with generalization, possibly due to its greedy nature.

Challenges:

- Small sample size (1,797) makes it easy to overfit.
- Some digits (like 3, 5, and 8) are visually similar, leading to confusion.

Improvements and Next Steps:

- **Use SVM with RBF kernel:** This adds non-linearity and may increase accuracy.
- **Cross-validation:** Instead of a single train/test split, use k-fold CV for better generalization estimates.
- **PCA (Principal Component Analysis):** Reduces dimensionality (64 pixels to fewer components), often improves performance.

- **Ensemble Models:**
 - **Random Forest** (many decision trees + averaging),
 - **Voting Classifier** (combines predictions from multiple models).